



CSE461

Introduction to Robotics Lab

Lab No. : 03

Group : 01

Section : 06

Semester : Summer_2025

Group members :

Azmari Sultana	22201949
B M Rauf	22201782
Sayeba Nasir	22201354

Submitted Date: 19-07-2025

Submitted to - CSDM

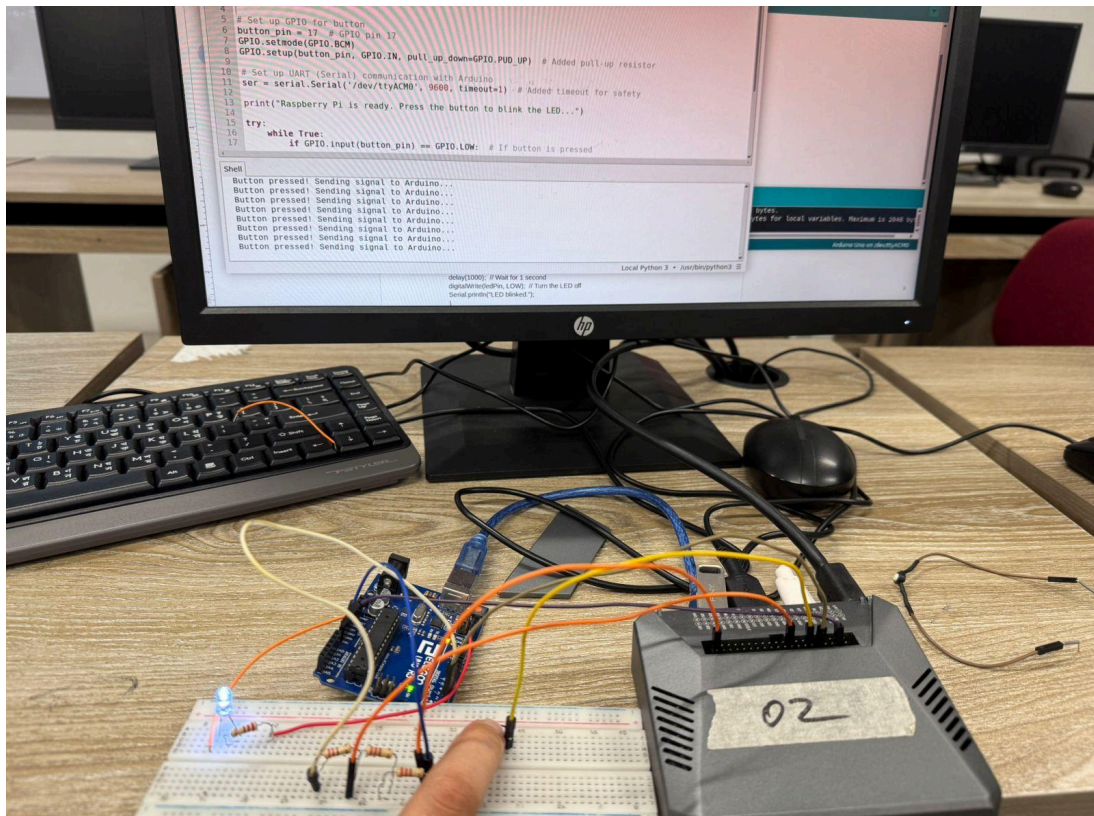
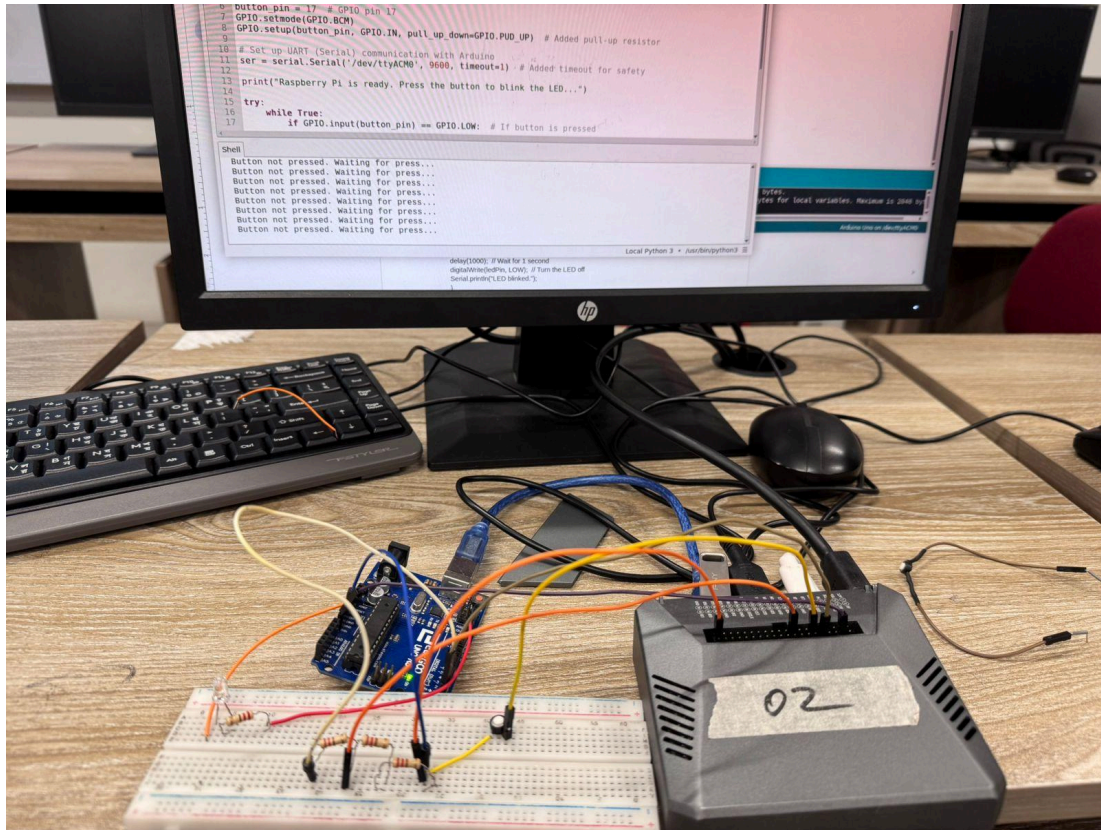
1. Objectives: The objective of this lab is to establish bidirectional serial communication (UART) between the two devices Arduino Uno and Raspberry Pi 4 and to control LED, it uses the communication to the Arduino through a push button connected to the Raspberry Pi. It addresses voltage compatibility between 3.3V Raspberry Pi and the 5V Arduino using a voltage divider. It also focuses on gaining hands-on experience in building and fixing systems that combine those two devices.

2. Equipments:

- Arduino Uno R3
- Raspberry Pi 4
- LED (Light Emitting Diode)
- Push-button switch
- Resistors: 220Ω (for LED), and resistors for voltage divider
- Breadboard
- Jumper wires

3. Experimental Setup:

The experimental setup attached interfacing an Arduino and a Raspberry Pi 4 using UART communication. The key components included a push button connected to the Raspberry Pi and LED connected to the Arduino with a 220Ω resistor. The Raspberry Pi was programmed to detect a button press on GPIO17 and send a serial command through its TX pin (GPIO14) to the Arduino's RX pin. After receiving the command, Arduino activated the LED for a set duration before turning it off. A voltage divider was implemented on the Raspberry Pi's RX line to safely interface the 5V output from the Arduino's TX pin with the 3.3V logic of the Raspberry Pi. To ensure proper communication, there was a shared common ground between the two systems. This configuration accommodated for reliable, bidirectional data exchange and demonstrated a practical example of hybrid embedded system design.



4. Code: (If Applicable)

```
# Raspberry PI Code
import RPi.GPIO as GPIO
import serial
import time

# Set up GPIO for button
button_pin = 17 # GPIO pin 17
GPIO.setmode(GPIO.BCM)
GPIO.setup(button_pin, GPIO.IN, pull_up_down=GPIO.PUD_UP) # pull-up resistor

# Set up UART (Serial) communication with Arduino
ser = serial.Serial('/dev/ttyACM0', 9600)

print("Raspberry Pi is ready. Press the button to blink the LED...")

try:
    while True:
        if GPIO.input(button_pin) == GPIO.LOW: # If button is pressed
            print("Button pressed! Sending signal to Arduino...")
            ser.write(b'1') # Send signal to Arduino to blink LED
            time.sleep(0.2) # Debounce delay
        else:
            print("Button not pressed. Waiting for press...")
            time.sleep(0.1) # Short delay to avoid excessive CPU usage
except KeyboardInterrupt:
    print("Program interrupted.")
finally:
    GPIO.cleanup()
    print("GPIO cleaned up. Exiting program.")

#Arduino Code
const int ledPin = 13; // LED pin

void setup() {
    Serial.begin(9600); // Initialize UART communication at 9600 baud
    pinMode(ledPin, OUTPUT); // Set the LED pin as an output
    Serial.println("Arduino is ready. Waiting for command...");
}

void loop() {
    if (Serial.available() > 0) {
        char command = Serial.read(); // Read the incoming byte
        Serial.print("Received: ");
        Serial.println(command);

        if (command == '1') {
            Serial.println("Button pressed: Blinking LED...");
            digitalWrite(ledPin, HIGH); // Turn the LED on
            delay(1000); // Wait for 1 second
            digitalWrite(ledPin, LOW); // Turn the LED off
            Serial.println("LED blinked.");
        }
    }
}
```

```
}  
}  
}
```

5. **Results (Output of the experiment):** In both directions, UART communication worked reliably and when we pressed the push button on Raspberry Pi triggered the LED on the Arduino to blink for 1 second. Additionally, a resistor based voltage divider which maintains voltage level shifting successfully protected Raspberry Pi from 5V signals.
6. **Discussions/Answers:** In this lab, we successfully established UART communication to control a LED using a push button between the Raspberry Pi and the Arduino Uno, and pressing the button on the Raspberry Pi triggered the LED connected to the Arduino to blink for a moment. It shows us how microcontrollers can work together by associating the input handling capabilities of the Raspberry Pi with the output control of the Arduino.

However, during the experiment, we encountered a problem with the Raspberry Pi code where the button state was always detected as pressed. This happened because the `pull_up_down=GPIO.PUD_UP` parameter was not initially added during GPIO setup, causing the GPIO input to float and read as LOW continuously.

After adding `pull_up_down=GPIO.PUD_UP`, the button was correctly detected, and the Raspberry Pi began sending the signal properly to the Arduino. Once the correction was made, the Arduino received the command as expected, and the LED blinked when the button was pressed.

Additionally, we learned that connecting UART pins before uploading code to the Arduino can cause uploading issues, as the TX/RX pins are shared between programming and serial communication. So, we uploaded code first, then connecting the UART pins helped to avoid the problem.

Overall, this lab teaches us the importance of hardware compatibility, correct wiring, proper coding and debugging techniques in robotics and IoT applications.